SIT315 - Transit Routing System

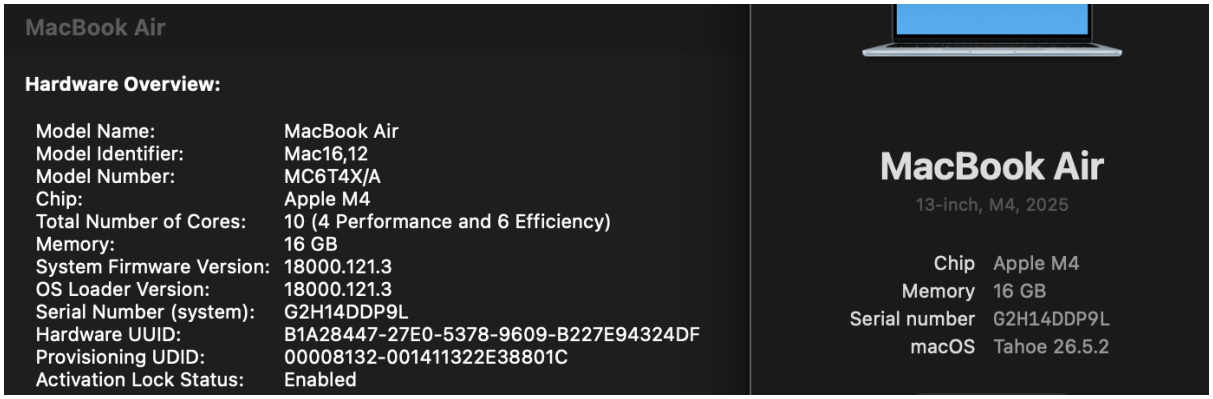
Mohammad Hassan Jamali – s225402184

Video Link: Link

GitHub Repository: https://github.com/hassan-jamali/vic_transit_routing

System Specifications

All implementations have been compiled and tested on a MacBook Air powered by an M4 chip (featuring 10 total CPU cores: 4 performance and 6 efficiency) with 16 GB of unified memory. The hardware specifications are detailed below.

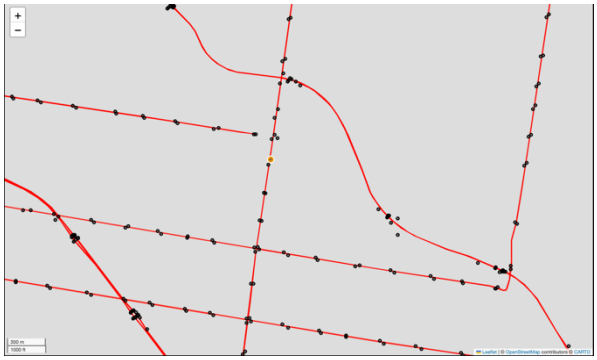


GTFS Data Architecture & Simulation

Firstly, the transit simulator relies on parsing real world General Transit Feed Specification (GTFS) data from the Victorian Transport Open Data Portal.

Since C++ is highly efficient at raw computation but less flexible with text parsing, the architecture splits into a Python data pipeline and a C++ routing system. In addition, the DataFile class utilises the pandas library to ingest stops.txt, stop_times.txt, and shapes.txt. Moreover, the Python wrapper pairs consecutive stops based on matching trip_id and sequential time boundaries to generate valid travel edges. Therefore, this data is exported into a lightweight transit_graph.json file, reserving the C++ TransitGraph class strictly for high-performance memory mapping and traversal.





Heuristic Design (Haversine Distance)

Firstly, the A* pathfinding algorithm relies on a heuristic function to estimate the remaining cost to the destination, preventing the algorithm from searching blindly. Since the transit network is mapped geographically, the simulator uses the Haversine formula in heuristic.cpp to calculate the great circle distance between two coordinates. In addition, by converting latitudes and longitudes into radians and applying the Earth's radius (6,371,000 meters), the heuristic provides a mathematically admissible straight-line distance.

heuristic.cpp > estimate_time(const TransitGraph &, const std::string &, const std::string &)

```cpp
#include "heuristic.h"
#include <cmath>

double Heuristic::calculate_distance(double latitude_one, double longitude_one, double latitude_two, double longitude_two) {
    // haversine distance in meters
    const double earth_radius_meters = 6371000.0;
    const double radians_per_degree = M_PI / 180.0;

    double delta_latitude = (latitude_two - latitude_one) * radians_per_degree;
    double delta_longitude = (longitude_two - longitude_one) * radians_per_degree;

    double haversine_a = std::sin(delta_latitude / 2) * std::sin(delta_latitude / 2) +
                         std::cos(latitude_one * radians_per_degree) * std::cos(latitude_two * radians_per_degree) *
                         std::sin(delta_longitude / 2) * std::sin(delta_longitude / 2);

    return earth_radius_meters * 2 * std::atan2(std::sqrt(haversine_a), std::sqrt(1 - haversine_a));
}

int Heuristic::estimate_time(const TransitGraph& graph, const std::string& current_id, const std::string& goal_id) {
    const auto& current_node = graph.nodes.at(current_id);
    const auto& goal_node = graph.nodes.at(goal_id);

    double distance_meters = calculate_distance(current_node.latitude, current_node.longitude, goal_node.latitude, goal_node.longitude);
    return static_cast<int>(distance_meters / 16.67);
}
```

Hybrid Parallel Architecture (Apple Metal)

Firstly, the heuristic distance calculations were reimplemented using Apple's Metal API to offload workloads from the CPU to the GPU. Since the architecture bridges C++ with Objective C++ (metal_heuristic.mm), it successfully compiles a Metal Shading Language (MSL) compute kernel (kBatchShaderSource) at runtime.

metal_heuristic.mm

```objc
#include "metal_heuristic.h"
#import <Metal/Metal.h>
#import <Foundation/Foundation.h>
#include <iostream>

namespace MetalHeuristic {

    // metal shader source embedded directly as a string
    static NSString* const kBatchShaderSource = @R"(
        #include <metal_stdlib>
        using namespace metal;

        kernel void calculate_haversine_batch(
            device const double* node_latitudes [[buffer(0)]],
            device const double* node_longitudes [[buffer(1)]],
            device const double* goal_latitudes [[buffer(2)]],
            device const double* goal_longitudes [[buffer(3)]],
            device double* heuristic_matrix [[buffer(4)]],
            constant uint& num_goals [[buffer(5)]],
            uint2 thread_position [[thread_position_in_grid]]
        ) {
            uint node_index = thread_position.x;
            uint goal_index = thread_position.y;

            double earth_radius_meters = 6371000.0;
            double degrees_to_radians = M_PI_F / 180.0;

            double latitude_1 = node_latitudes[node_index] * degrees_to_radians;
            double longitude_1 = node_longitudes[node_index] * degrees_to_radians;
            double latitude_2 = goal_latitudes[goal_index] * degrees_to_radians;
            double longitude_2 = goal_longitudes[goal_index] * degrees_to_radians;

            double delta_latitude = latitude_2 - latitude_1;
            double delta_longitude = longitude_2 - longitude_1;

            double a = sin(delta_latitude / 2.0) * sin(delta_latitude / 2.0) +
                       cos(latitude_1) * cos(latitude_2) *
                       sin(delta_longitude / 2.0) * sin(delta_longitude / 2.0);

            double c = 2.0 * atan2(sqrt(a), sqrt(1.0 - a));

            uint output_index = node_index * num_goals + goal_index;
            heuristic_matrix[output_index] = earth_radius_meters * c;
        }
    )";
```

In addition, the system allocates shared MTLBuffer memory blocks to pass array pointers for node and goal coordinates directly to the GPU.

```
ize_t node_buffer_size = node_count * sizeof(double);
ize_t goal_buffer_size = goal_count * sizeof(double);
ize_t matrix_buffer_size = node_count * goal_count * sizeof(double);

/ firstly, allocate shared memory buffers for nodes and goals
d<MTLBuffer> buffer_node_latitudes = [ctx.device newBufferWithBytes:node_latitudes.data() length:node_buffer_size options:MTLResourceStorageModeShared];
d<MTLBuffer> buffer_node_longitudes = [ctx.device newBufferWithBytes:node_longitudes.data() length:node_buffer_size options:MTLResourceStorageModeShared];
d<MTLBuffer> buffer_goal_latitudes = [ctx.device newBufferWithBytes:goal_latitudes.data() length:goal_buffer_size options:MTLResourceStorageModeShared];
d<MTLBuffer> buffer_goal_longitudes = [ctx.device newBufferWithBytes:goal_longitudes.data() length:goal_buffer_size options:MTLResourceStorageModeShared];
d<MTLBuffer> buffer_heuristic_matrix = [ctx.device newBufferWithLength:matrix_buffer_size options:MTLResourceStorageModeShared];
```

Moreover, a compute pipeline state is cached, allowing the GPU to spawn thousands of concurrent threads mapped via the [[thread_position_in_grid]] parameter. Therefore, once the command buffer commits via waitUntilCompleted, the engine immediately retrieves a fully populated matrix of distances.

Performance Profiling (Hybrid Architecture Evaluation)

Firstly, profiling was conducted to evaluate the trade-offs of the hybrid CPU / GPU architecture across varying workloads. In addition, single commuter routing was benchmarked across increasing route lengths and network disruptions (up to 2,000 dynamically blocked stations). Since single target A* pathfinding relies on a sequential min-heap priority queue, the CPU vastly outperformed the GPU, completing complex detours in 0.008 to 0.59 milliseconds while the GPU required around 52 to 55 milliseconds.

```
test_configs = [
    {"waypoints": 3,  "blocks": 1},
    {"waypoints": 5,  "blocks": 50},
    {"waypoints": 7,  "blocks": 250},
    {"waypoints": 10, "blocks": 500},
    {"waypoints": 15, "blocks": 1000},
    {"waypoints": 20, "blocks": 2000}
]

results = []

for config in test_configs:
    w = config["waypoints"]
    b = config["blocks"]

    # mute the print spam
    with contextlib.redirect_stdout(io.StringIO()):
        route, time_sec, primary_block = transit_data.get_valid_multi_route(num_waypoints=w, seed=10)

    blocks = [primary_block]

    if b > 1:
        all_stops = transit_data.stops['stop_id'].tolist()
        safe_stops = [stop for stop in all_stops if stop not in route]
        # ensure we don't try to block more stops than exist
        actual_blocks_to_add = min(b - 1, len(safe_stops))
        extra_blocks = random.sample(safe_stops, actual_blocks_to_add)
        blocks.extend(extra_blocks)

    command = ["--time", time_sec, "--route", *route, "--block", *blocks]

    results.append({
        "waypoints": w,
        "blocks": b,
        "cpu time (ms)": get_time(command, use_gpu=False),
        "gpu time (ms)": get_time(command, use_gpu=True)
    })

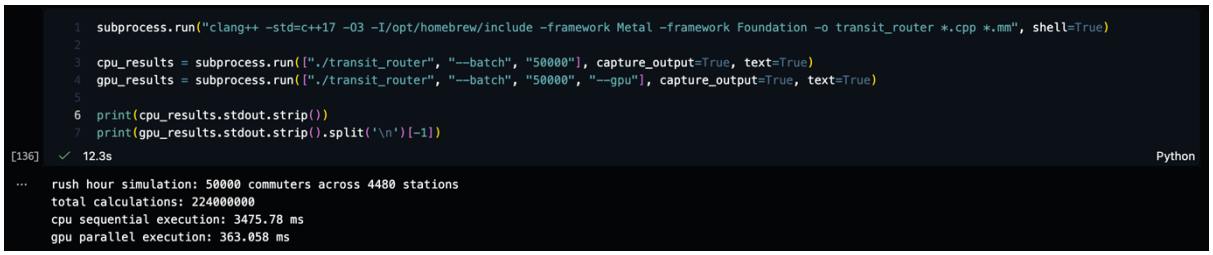
df = pd.DataFrame(results)
print(df.to_string(index=False))
```

[138] ✓ 24.3s Python

waypoints	blocks	cpu time (ms)	gpu time (ms)
3	1	0.234917	236.985417
5	50	0.467083	52.265166
7	250	0.460708	58.554792
10	500	0.685167	53.211667
15	1000	0.569167	56.946917
20	2000	0.008417	55.777292

Moreover, this latency discrepancy is driven by macOS Metal driver overhead, which incurs costs for pipeline synchronisation (waitUntilCompleted) and shared buffer allocation on every independent query. Therefore, this demonstrates Amdahl's Law in practice: sequential algorithms with high synchronisation requirements cannot exploit SIMD/GPU parallelism and execute significantly faster within the CPU's local L1/L2 cache.

In addition, when shifting from sequential graph traversal (AStarRouter::find_route) to batch heuristic precalculation (simulating 50,000 commuters across 4,480 stations), the workload converted into a fully decoupled, parallel matrix operation. Since the GPU bypassed sequential branching to run 224 million calculations across thousands of concurrent threads independently, the Apple Metal kernel (calculate_batch_distances) completed the batch in 338.2 milliseconds compared to the CPU's 3,198.8 milliseconds. Therefore, the hybrid pipeline successfully demonstrated nearly a 10x throughput advantage once thread saturation masked driver overhead.



Stress Test

Firstly, an extreme stress test was configured by ingesting all 6 GTFS transport modes (Regional Train, Metropolitan Train, Metropolitan Tram, Bus, and Coach) combined with a 50,000 commuter batch request to determine system breaking points. In addition, loading all 6 modes via DataFile(transport_modes=[1, 2, 3, 4, 5, 6]) scaled the transit graph from 4,480 stations to tens of thousands of combined stops and transit edges.

Since the Metal compute kernel constructs a dense Haversine distance matrix of dimensions $\text{node_count} \times \text{commuter_count}$, the required memory allocation scaled to:

$$\text{matrix_buffer_size} = \text{node_count} \times 50{,}000 \times 8 \text{ bytes}$$

Moreover, on a graph exceeding 30,000 stops, this single contiguous matrix required over 12 GB of uncompressed floating-point memory, operating alongside graph adjacency structures within the 16 GB unified memory architecture. Since [ctx.device newBufferWithLength:options:] was unable to allocate the requested contiguous buffer under macOS memory pressure, the allocation returned nil, triggering an unhandled memory fault and an immediate crash, recorded as a Did Not Finish (DNF).

Transit Routing System

A C++ transit routing system which uses an A* pathfinding algorithm and accelerated using Apple Metal (GPU) for parallel Haversine distance computations. It supports single and multi modal public transport networks ingested directly from GTFS datasets.

Repository File Structure

```
├── a_star_router.cpp       # A* pathfinding core logic implementation
├── a_star_router.h
├── heuristic.cpp           # CPU fallback distance heuristic calculations
├── heuristic.h
├── main.cpp                # CLI entry point supporting routing and benchmarking
├── main.ipynb              # Jupyter notebook for data ingestion, cleaning, and mapping
├── metal_heuristic.h
├── metal_heuristic.mm      # Objective C++ Metal compute kernel and context manager
├── requirements.txt        # Python package dependencies
├── transit_graph.cpp       # Network graph loader and adjacency list structures
└── transit_graph.h
```

(Note: Large datasets, compiled binaries like transit_router, and JSON outputs are excluded from the repository and are generated locally.)

Prerequisites & Libraries

The data cleaning pipeline relies on **pandas** and **geopandas** for spatial manipulation, along with **matplotlib** and **shapely** for visualization. The router requires a C++17 compiler and Apple Metal hardware.

Install all required Python packages using:

```
pip install -r requirements.txt
```

GTFS Data Setup

Download the desired GTFS schedule datasets from the Victorian Transport Open Data Portal. Place the unzipped folders into a local `./gtfs/` directory organized by their transport mode number:

- **1**: Regional Train
- **2**: Metropolitan Train
- **3**: Metropolitan Tram

- **4**: Myki Bus (Metro Bus and Regional Town Bus)
- **5**: Regional Coach
- **6**: Regional Bus
- **10**: Interstate
- **11**: SkyBus

Data Preparation (`main.ipynb`)

Before routing, prepare the raw GTFS data. Open `main.ipynb` to ingest, clean, and export your transit networks into a structured `transit_graph.json` file using the provided `DataFile` class.

1. Generate the Network Graph

Initialize single or multi modal feeds (e.g., combining train and tram feeds) directly in the notebook:

```
# combine metropolitan train (2) and tram (3) feeds into one unified graph
transit_data = DataFile(transport_modes=[2, 3])
transit_data.clean()
transit_data.prepare()
transit_data.export_json("transit_graph.json")
```

2. Finding Station IDs

Because the C++ router uses numeric Stop IDs rather than text names, search for the exact IDs interactively in Python using case insensitive queries before running terminal commands:

```
# find station IDs matching a specific name query
stations = transit_data.find_station("Central")
print(stations)
# Output displays the corresponding stop_id needed for the CLI (e.g., 10922)
```

Compilation & CLI Commands

Compile the C++ router, the Objective-C++ Metal bridge, and the GPU compute shaders into a single executable:

```
clang++ -std=c++17 -O3 -I/opt/homebrew/include -framework Metal -framework Foundation -o
transit_router *.cpp *.mm
```

Available Commands

Once compiled, execute queries via the terminal. *(Note: The Stop IDs below are randomised examples. Use the `find_station` method in your Jupyter Notebook to get valid IDs for your specific dataset).*

- **Standard CPU Route Search:** Computes the shortest path using the CPU baseline.

```
./transit_router --route 4192 8274 --time 18000
```

- **GPU Accelerated Route Search:** Offloads distance heuristic calculations to Apple Metal.

```
./transit_router --gpu --route 4192 8274 --time 18000
```

- **Dynamic Detours (Station Blocking):** Sever graph segments by blocking specific station IDs to force the algorithm to find an alternative detour.

```
./transit_router --gpu --route 4192 8274 --time 18000 --block 5931
```

- **Internal Performance Benchmarking:** Runs a 100 iteration benchmark loop directly inside the C++ binary to test pure hardware speed, bypassing Python process-spawning overhead.

```
./transit_router --benchmark --gpu --route 4192 8274 --time 18000
```

Importing Libraries

We'll be importing the pandas and geopandas library for data cleaning/preparation and matplotlib, shapely for data visualisation.

To install the following libraries, you can install them using the `requirements.txt` file by running the following command:

```
pip install -r requirements.txt
```

```
import subprocess
import os
import random
import contextlib
import io
import pandas as pd
import geopandas as gpd
import numpy as np
import matplotlib.pyplot as plt
from shapely.geometry import LineString
import json
from typing import Any
```

We then created a class for the Data File generation that does the preparation/cleaning and JSON export for you. The only input you do need is the transport mode number which will tell you which transport mode corresponds to what depending on the GTFS dataset that you have installed. The available branches are as follows:

- **1** (Regional Train)
- **2** (Metropolitan Train)
- **3** (Metropolitan Tram)
- **4** Myki Bus (Metro Bus and Regional Town Bus)
- **5** (Regional Coach)
- **6** (Regional Bus)
- **10** (Interstate)
- **11** (SkyBus)

Source: GTFS Schedule

```
class DataFile:
    def __init__(self, transport_mode: int | list[int]) -> None:
```

```
        # normalise the input to handle single mode or multiple modes
        modes = [transport_mode] if isinstance(transport_mode, int)
else transport_mode
        stops_list = []
        stop_times_list = []
        shapes_list = []
        # for each mode, set up the specific data frames
        for mode in modes:
            base = f'./gtfs/{mode}/google_transit/'
            stops_df = pd.read_csv(base + 'stops.txt')
            stop_times_df = pd.read_csv(base + 'stop_times.txt',
low_memory=False)
            shapes_df = pd.read_csv(base + 'shapes.txt')
            # prefix ids to prevent collisions across different feeds
            stop_times_df['trip_id'] = f"{mode}_" +
stop_times_df['trip_id'].astype(str)
            shapes_df['shape_id'] = f"{mode}_" +
shapes_df['shape_id'].astype(str)

            stops_list.append(stops_df)
            stop_times_list.append(stop_times_df)
            shapes_list.append(shapes_df)
        # combine feeds into one unified master dataframes
        self.stops = pd.concat(stops_list,
ignore_index=True).drop_duplicates(subset=['stop_id'])
        self.stop_times = pd.concat(stop_times_list,
ignore_index=True)
        self.shapes = pd.concat(shapes_list, ignore_index=True)
        self.edges = None

    def clean(self) -> None:
        # filter and rename columns
        self.stops = self.stops[['stop_id', 'stop_name', 'stop_lat',
'stop_lon']].rename(
            columns={'stop_lat': 'stop_latitude', 'stop_lon':
'stop_longitude'}
        )

        # dry helpers for column ops
        clean_id = lambda s:
s.astype(str).str.split('.').str[0].str.strip()
        to_sec = lambda s:
pd.to_timedelta(s).dt.total_seconds().astype(int)

        # apply cleans
        self.stops['stop_id'] = clean_id(self.stops['stop_id'])
        self.stop_times['stop_id'] =
clean_id(self.stop_times['stop_id'])
```

```
        # convert times
        self.stop_times['departure_sec'] =
to_sec(self.stop_times['departure_time'])
        self.stop_times['arrival_sec'] =
to_sec(self.stop_times['arrival_time'])

    def prepare(self) -> None:
        # sort and pair consecutive stops
        df = self.stop_times.sort_values(by=['trip_id',
'stop_sequence']).reset_index(drop=True)

        df['to_stop_id'] = df['stop_id'].shift(-1)
        df['next_arrival_sec'] = df['arrival_sec'].shift(-1)
        df['next_trip_id'] = df['trip_id'].shift(-1)

        # filter valid boundaries and calc durations
        self.edges = df[df['trip_id'] ==
df['next_trip_id']].copy().rename(
            columns={'stop_id': 'from_stop_id'}
        )
        self.edges['travel_time_sec'] = self.edges['next_arrival_sec']
- self.edges['departure_sec']

    def export_json(self, output_filename: str = "transit_graph.json")
-> None:
        # build nodes dictionary
        nodes = {
            row['stop_id']: {
                "name": row['stop_name'],
                "latitude": float(row['stop_latitude']),
                "longitude": float(row['stop_longitude'])
            } for _, row in self.stops.iterrows()
        }

        # build edges dictionary
        edges = {}
        for _, row in self.edges.iterrows():
            f_node, t_node = str(row['from_stop_id']),
str(row['to_stop_id'])

            # create nested dicts cleanly
            edges.setdefault(f_node, {}).setdefault(t_node,
[]).append({
                "trip_id": row['trip_id'],
                "departure_time": int(row['departure_sec']),
                "arrival_time": int(row['next_arrival_sec']),
                "travel_time": int(row['travel_time_sec'])
            })
        # save
```

```
        with open(output_filename, 'w') as f:
            json.dump({"nodes": nodes, "edges": edges}, f, indent=2)

    @property
    def get_stops(self) -> tuple[pd.DataFrame, pd.DataFrame,
pd.DataFrame]:
        return self.stops, self.stop_times, self.shapes

    def _get_stops_gdf(self) -> gpd.GeoDataFrame:
        return gpd.GeoDataFrame(
            self.stops,
            geometry=gpd.points_from_xy(self.stops['stop_longitude'],
self.stops['stop_latitude']),
            crs="EPSG:4326"
        )

    def find_station(self, search_query: str) -> pd.DataFrame:
        # find stations matching the query string (case insensitive)
        matches =
self.stops[self.stops['stop_name'].str.contains(search_query,
case=False, na=False)]
        return matches[['stop_id',
'stop_name']].reset_index(drop=True)

    def get_valid_trips(self, start_query: str, goal_query: str,
min_time_sec: int = 18000) -> pd.DataFrame:
        # get matching stop ids
        starts = self.find_station(start_query)['stop_id']
        goals = self.find_station(goal_query)['stop_id']
        # filter stop times
        stop = self.stop_times
        start_trips = stop[stop['stop_id'].isin(starts)]
        goal_trips = stop[stop['stop_id'].isin(goals)]
        # merge on shared trips
        valid = pd.merge(start_trips, goal_trips, on='trip_id',
suffixes=('_start', '_goal'))
        # enforce correct direction and minimum departure time
        valid = valid[
            (valid['stop_sequence_start'] <
valid['stop_sequence_goal']) &
            (valid['departure_sec_start'] >= min_time_sec)
        ]

        # return clean sorted options
        columns = ['trip_id', 'stop_id_start', 'stop_id_goal',
'departure_sec_start']
        return
valid[columns].sort_values('departure_sec_start').reset_index(drop=Tru
e)
```

```
    def get_valid_route(self, trip_id: str = None, min_stops: int = 5,
seed: int = None) -> tuple[str, str, str]:
        # set seed for reproducible randomness (if none, it stays
truly random)
        random.seed(seed)
        # fallback to random if no specific trip is provided
        if not trip_id:
            trip_counts = self.stop_times['trip_id'].value_counts()
            valid_trips = trip_counts[trip_counts >= min_stops].index
            trip_id = random.choice(valid_trips.tolist())
        # get all stops for this trip sorted by sequence
        trip_stops = self.stop_times[self.stop_times['trip_id'] ==
trip_id].sort_values('stop_sequence')
        if trip_stops.empty:
            print(f"error: trip {trip_id} not found")
            return "", "", ""
        # extract start and end points
        first = trip_stops.iloc[0]
        last = trip_stops.iloc[-1]
        # get readable names
        stops_idx = self.stops.set_index('stop_id')
        start_name = stops_idx.at[first['stop_id'], 'stop_name']
        goal_name = stops_idx.at[last['stop_id'], 'stop_name']

        print(f"route found ({trip_id}):")
        print(f"{start_name} -> {goal_name}")

        # return as strings for subprocess
        return str(first['stop_id']), str(last['stop_id']),
str(first['departure_sec'])

    def get_valid_multi_route(self, num_waypoints: int = 3, min_stops:
int = 15, seed: int = None) -> tuple[list[str], str, str]:
        random.seed(seed)

        # force the trip to have more stops than our required
waypoints
        actual_min_stops = max(min_stops, num_waypoints + 5)

        trip_counts = self.stop_times['trip_id'].value_counts()
        valid_trips = trip_counts[trip_counts >=
actual_min_stops].index
        trip_id = random.choice(valid_trips.tolist())

        trip_stops = self.stop_times[self.stop_times['trip_id'] ==
trip_id].sort_values('stop_sequence')

        indices = [int(i * (len(trip_stops) - 1) / (num_waypoints -
1)) for i in range(num_waypoints)]
        selected_stops = trip_stops.iloc[indices]
```

```
        route_nodes = selected_stops['stop_id'].astype(str).tolist()
        time_seconds = str(trip_stops.iloc[0]['departure_sec'])

        # bulletproof fallback to find a node that is not a waypoint
        available_indices = [i for i in range(len(trip_stops)) if i
not in indices]
        block_index = available_indices[len(available_indices) // 2]
        block_node = str(trip_stops.iloc[block_index]['stop_id'])

        stops_idx = self.stops.set_index('stop_id')
        names = [stops_idx.at[n, 'stop_name'] for n in route_nodes]
        block_name = stops_idx.at[block_node, 'stop_name']

        print(f"multi route found ({trip_id}):")
        print(" -> ".join(names))
        print(f"suggested node to block: {block_name} ({block_node})")

        return route_nodes, time_seconds, block_node

    def draw_transit(self, interactive: bool = False) -> Any:
        # build shape lines
        shapes_sorted = self.shapes.sort_values(by=['shape_id',
'shape_pt_sequence'])
        shape_lines = shapes_sorted.groupby('shape_id').apply(
            lambda df: LineString(list(zip(df['shape_pt_lon'],
df['shape_pt_lat'])))
        ).reset_index(name='geometry')

        gdf_shapes = gpd.GeoDataFrame(shape_lines,
geometry='geometry', crs="EPSG:4326")
        gdf_stops = self._get_stops_gdf()

        if interactive:
            # render folium map
            m = gdf_shapes.explore(color="red",
style_kwds=dict(weight=2, opacity=0.6), tiles="CartoDB positron")
            gdf_stops.explore(m=m, color="black",
marker_type="circle_marker", marker_kwds=dict(radius=3, fill=True),
tooltip=["stop_id", "stop_name"])
            return m

        # render static plot
        fig, ax = plt.subplots(figsize=(12, 10))
        gdf_shapes.plot(ax=ax, color='red', linewidth=1, alpha=0.5,
label='true track routes')
        gdf_stops.plot(ax=ax, color='black', markersize=6, zorder=3,
label='stations')
        ax.set_axis_off()
```

```
        plt.legend()
        plt.show()

    def verify_route(self, route_json_path: str, interactive: bool =
False) -> Any:
        # load cpp result
        with open(route_json_path, 'r') as f:
            route_data = json.load(f)

        if not route_data.get("found"):
            print("no route found")
            return

        execution_time_ms =
route_data.get("execution_time_microseconds", 0)
        print(f"a* execution time: {execution_time_ms:.1f} µs
({execution_time_ms / 1000.0:.3f} ms)")

        # extract path coordinates
        stops_idx = self.stops.set_index('stop_id')
        valid_nodes = [n for n in route_data["path"] if n in
stops_idx.index]

        if len(valid_nodes) < 2:
            print("invalid route length")
            return
        # print the sequence to see the direction of travel
        print("route itinerary:")
        for i, node in enumerate(valid_nodes):
            name = stops_idx.at[node, 'stop_name']
            print(f"{i + 1}. {name}")

        coords = [(stops_idx.at[n, 'stop_longitude'], stops_idx.at[n,
'stop_latitude']) for n in valid_nodes]
        gdf_route = gpd.GeoDataFrame([{'geometry':
LineString(coords)}], crs="EPSG:4326")

        if interactive:
            # layer route on base map
            m = self.draw_transit(interactive=True)
            gdf_route.explore(m=m, color="cyan", style_kwds={"weight":
6, "opacity": 1.0}, name="a* route")
            return m

        # layer route on static plot
        fig, ax = plt.subplots(figsize=(12, 10))
        self._get_stops_gdf().plot(ax=ax, color='lightgray')
        gdf_route.plot(ax=ax, color='blue', label='a* route')
        ax.set_axis_off()
```

```
        plt.legend()
        plt.show()
```

Example output in this case, we would be the Metropolitan Tram and Train.

```
transit_data = DataFile([2, 3])
transit_data.clean()
transit_data.prepare()

transit_data.export_json("transit_graph.json")

start_id, goal_id, time_seconds =
transit_data.get_valid_route(seed=10)

result = subprocess.run(
    ["./transit_router", start_id, goal_id, time_seconds],
    capture_output=True,
    text=True
)

if result.stdout: print(result.stdout.strip())
if result.stderr: print("errors:", result.stderr.strip())

# verify computed route over transit map
route_map = transit_data.verify_route("route_result.json",
interactive=True)
route_map

route found (2_02-CGB--63-T6-5823):
Flinders Street Station -> Craigieburn Station
route exported to route_result.json
a* execution time: 392.0 µs (0.392 ms)
route itinerary:
1. Flinders Street Station
2. Parliament Station
3. Melbourne Central Station
4. Flagstaff Station
5. North Melbourne Station
6. Kensington Station
7. Newmarket Station
8. Ascot Vale Station
9. Moonee Ponds Station
10. Essendon Station
11. Glenbervie Station
12. Strathmore Station
13. Pascoe Vale Station
14. Oak Park Station
15. Glenroy Station
16. Jacana Station
17. Broadmeadows Station
```

```
18. Coolaroo Station
19. Roxburgh Park Station
20. Craigieburn Station

<folium.folium.Map at 0x12f586e40>
```

```
def get_time(command, use_gpu=False):
    cmd = ["./transit_router", "--gpu"] + command if use_gpu else
["./transit_router"] + command
    subprocess.run(cmd, capture_output=True)
    with open("route_result.json", 'r') as f:
        return json.load(f).get("execution_time_microseconds", 0) /
1000.0

results = []

for waypoints in [3, 5, 7, 10]:
    route, time_sec, block =
transit_data.get_valid_multi_route(num_waypoints=waypoints, seed=10)
    command = ["--time", time_sec, "--route", *route, "--block",
block]

    results.append({
        "waypoints": waypoints,
        "blocks": 1,
        "cpu time (ms)": get_time(command, use_gpu=False),
        "gpu time (ms)": get_time(command, use_gpu=True)
    })

df = pd.DataFrame(results)
print(df.to_string(index=False))
```

```
multi route found (2_02-CGB--63-T6-5823):
Flinders Street Station -> Essendon Station -> Craigieburn Station
suggested node to block: Glenbervie Station (15542)
multi route found (2_02-CGB--63-T6-5823):
Flinders Street Station -> North Melbourne Station -> Essendon Station
-> Glenroy Station -> Craigieburn Station
suggested node to block: Glenbervie Station (15542)
multi route found (2_02-CGB--63-T6-5823):
Flinders Street Station -> Flagstaff Station -> Newmarket Station ->
Essendon Station -> Pascoe Vale Station -> Jacana Station ->
Craigieburn Station
suggested node to block: Glenbervie Station (15542)
multi route found (2_02-CGB--63-T6-5823):
Flinders Street Station -> Melbourne Central Station -> North
Melbourne Station -> Newmarket Station -> Moonee Ponds Station ->
Glenbervie Station -> Pascoe Vale Station -> Glenroy Station ->
Broadmeadows Station -> Craigieburn Station
suggested node to block: Strathmore Station (15540)
```

```
 waypoints  blocks  cpu time (ms)  gpu time (ms)
         3       1       1.541916     320.005000
         5       1       1.200875      81.068209
         7       1       0.324875      80.791667
        10       1       0.249875     165.794834
```

```
test_configs = [
    {"waypoints": 3,  "blocks": 1},
    {"waypoints": 5,  "blocks": 50},
    {"waypoints": 7,  "blocks": 250},
    {"waypoints": 10, "blocks": 500},
    {"waypoints": 15, "blocks": 1000},
    {"waypoints": 20, "blocks": 2000}
]

results = []

for config in test_configs:
    w = config["waypoints"]
    b = config["blocks"]

    # mute the print spam
    with contextlib.redirect_stdout(io.StringIO()):
        route, time_sec, primary_block =
transit_data.get_valid_multi_route(num_waypoints=w, seed=10)

    blocks = [primary_block]

    if b > 1:
        all_stops = transit_data.stops['stop_id'].tolist()
        safe_stops = [stop for stop in all_stops if stop not in route]
        # ensure we don't try to block more stops than exist
        actual_blocks_to_add = min(b - 1, len(safe_stops))
        extra_blocks = random.sample(safe_stops, actual_blocks_to_add)
        blocks.extend(extra_blocks)

    command = ["--time", time_sec, "--route", *route, "--block",
*blocks]

    results.append({
        "waypoints": w,
        "blocks": b,
        "cpu time (ms)": get_time(command, use_gpu=False),
        "gpu time (ms)": get_time(command, use_gpu=True)
    })

df = pd.DataFrame(results)
print(df.to_string(index=False))
```

```
 waypoints  blocks  cpu time (ms)  gpu time (ms)
         3       1       0.866541      92.815958
         5      50       0.305625      75.402208
         7     250       0.950042      73.883459
        10     500       1.200333      56.945750
        15    1000       0.403916     196.400750
        20    2000       0.022958     199.402583
```

```
subprocess.run("clang++ -std=c++17 -O3 -I/opt/homebrew/include -
framework Metal -framework Foundation -o transit_router *.cpp *.mm",
shell=True)

cpu_results = subprocess.run(["./transit_router", "--batch", "50000"],
capture_output=True, text=True)
gpu_results = subprocess.run(["./transit_router", "--batch", "50000",
"--gpu"], capture_output=True, text=True)

print(cpu_results.stdout.strip())
print(gpu_results.stdout.strip().split('\n')[-1])
```

```
rush hour simulation: 50000 commuters across 4480 stations
total calculations: 224000000
cpu sequential execution: 4782.84 ms
gpu parallel execution: 618.653 ms
```

```
transit_data = DataFile(transport_mode=[2])
transit_data.clean()
transit_data.prepare()

route, time_sec, block_node =
transit_data.get_valid_multi_route(num_waypoints=3, seed=62)

origin = route[0]
waypoint = route[1]
destination = route[2]

print(f"./transit_router --route {origin} {destination} --time
{time_sec}")

print(f"./transit_router --route {origin} {waypoint} {destination} --
time {time_sec} --block {block_node}")

print(f"./transit_router --gpu --route {origin} {waypoint}
{destination} --time {time_sec} --block {block_node}")

print("./transit_router --batch 50000")
print("./transit_router --batch 50000 --gpu")
```

```
multi route found (2_02-SUY--1-UU-Z109):
Town Hall Station -> Tottenham Station -> Sunbury Station
suggested node to block: Sunshine Station (26531)
```

```
./transit_router --route 26554 15352 --time 66840
./transit_router --route 26554 15513 15352 --time 66840 --block 26531
./transit_router --gpu --route 26554 15513 15352 --time 66840 --block
26531
./transit_router --batch 50000
./transit_router --batch 50000 --gpu

route_map = transit_data.verify_route("route_result.json",
interactive=True)
route_map

no route found
```